ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Parallel Programming
Assignment 8: The Java Memory Model

Overview

In this series of exercises we will introduce the Java memory model.

The focus here is on building an understanding why memory models are useful, and why an overly restrictive memory model (although it makes it easier to reason about a piece of code) is undesirable.

If you would like some additional reading on this topic we recommend to read the article "You Don't Know Jack about Shared Variables or Memory Models" by Adve and Boehm (you will not have any benefit in the exam if you do, but sometimes it is nice being able to relate what you learn in a lecture to what people in the real world are discussing).

1 Getting Prepared

This exercise does not come with any code template.

2 Motivation for Memory Models

In this section you will look at pieces of code and check if the outcome of their execution depends on the interleaving of threads or not. You then connect these theoretical exercises to practice by checking if you also observe the same behaviour in a Java program.

2.1 When are interleavings bad?

Assume there are two Java threads, sharing the variables v, w, x, y and z. The variables r1 and r2 are private. Assume the code these two threads are executing looks as follows:

Thread 1	Thread 2
x=23;	y=42;
r1 = x;	r2 = y;
v = r1;	w = r2;
z = 2;	

Is the result (the values of the private variables after both threads finish the execution) always the same or could it depend on the order in which the threads are scheduled?

Thread 1	Thread 2
x = 23;	y = 42;
r1 = x;	r2 = y;
v = r1;	w = r2;
y = 2;	z = 2;

How about this piece of code, is the result always the same or does it depend on the scheduling of the threads?

2.2 Interleavings in practice

In task 2.1 we looked at code, analyzing it within our brain. Use your knowledge on programming with Java to turn the code samples from Task 2.1 into real running code. Run your code multiple times, can you confirm your findings from Task before? There is no skeleton provided for this task, the point here is to make a connection between theory and practice, not to copy-paste a code-snippet into the right place.

Hint: If this does not produce the output you expected do not get discouraged, incorrectly synchronized programs are technically not valid Java programs so they often do not behave as we think — differentiating between correct and incorrect synchronization is one of the goals of this class. What you might try is marking your shared variables as volatile.

3 Building Blocks of the Java Memory Model

3.1 Relations

A relation is a mathematical concept defined over elements of a set. For example over the set of natural numbers we know the relation “is less than”. A binary (concerning two elements) relation R over a set S can be expressed as ordered pairs over $S \times S$. Instead of $(s_0, s_1) \in R$ we often write $s_0 R s_1$.

Relations can have different properties, for example:

Symmetry: $\forall s_0, s_1 \in S : s_0 R s_1 \rightarrow s_1 R s_0$
Reflexivity: $\forall s_0 \in S : s_0 R s_0$
Transitivity: $\forall s_0, s_1, s_2 \in S : s_0 R s_1 \wedge s_1 R s_2 \rightarrow s_0 R s_2$

Show that the relation ”beats” in the game of rock-paper-scissors is not transitive.

3.2 Transitive Closure

For a relation R we call the smallest relation which contains R and is transitive the transitive closure R^+ of R.

For the set $S : \{a, b, c, d, e, f\}$ and the relation X over $S : (a, b), (c, d), (a, c), (e, f)$ give the transitive closure X^+.

3.3 Program Order

Program order is the relation given to statement executions by the source code. We write $S1 \rightarrow S2$ to express: if $S1$ and $S2$ are executed they are executed in the order $S1$ immediately followed by $S2$. Note that in a loop, each traversal of the loop body would generate new statements. For the (sequential) code below, which of the statements are in program order? What is the transitive closure of the program order relation on the piece of code below?

```
S1: a=23;
S2: x=3;
S3: if (x==3) {
S4:  b += 1;
    } else {
S5:   b *= 2;
S6:   x = 0;
    }
S7: x=4;
```

Note that we define program order only for sequential blocks of code! Due to the randomness of interleavings we cannot really say if a statement executed by thread A will be followed by another statement of thread A or a statement of thread B.

3.4 Is Program Order Enough?

Program order is great to specify the behaviour of a sequential block of code. A very informal memory model for sequential code could be "the program should appear as if all statements have been executed in program order". One way to achieve that would be to simply execute every statement when encountered, without doing any optimizations. However, such a compiler would be wasteful. Imagine a code snippet such as

```
int funca() {
  for (int i=0; i<9999; i++) {
    b=3;
  }
  return b;
}
```

How could a compiler optimize funca() so that it still behaves as intended to an "observer" who is simply calling the function and using the return value?

Note that it is important to be clear which effect of the program are observable. Here we only look at the return value. Probably printed values should also not change. But if the code uses shared variables then suddenly another thread could observe how these change their values! We see that simply relying on program order is not enough, either we disallow all modifications (bad), or we additionally need a set of observable actions and must ensure that these actions "look like" everything was done in program order.

3.5 Happens Before

When reasoning about the behaviour of parallel code, it is often useful to define a ”happens before” relation as well. For the piece of code below (executed by two threads) look at each given output and draw arrows which indicate in which order statements must have been executed to produce this output. Your ”happens before” arrows must contain the program order relation for each threads code. Initially the shared variables x and y are 0.

Thread 1	Thread 2
x = 1;	y = 1;
r1 = y;	r2 = x;

Output 1: r1=0, r2=1.
Output 2: r1=1, r2=1.

4 Javas Memory Model

Javas memory model defines a group of actions, called synchronization actions, which when performed by a thread, relate to other actions within this group even if performed by other threads. They form a new relation, the ”synchronizes with” relation. Synchronization actions include locks, unlocks, reads of volatile variables, and writes to volatile variables (we ignore others here).

A read of a volatile variable synchronizes with the last previous write of this variable. A lock of a mutex synchronizes with the last previous unlock, an unlock with the last previous lock.

The transitive closure of program order (PO) and synchronizes-with order (SW) forms the happens-before order. Values of variables must be consistent with the happens-before order (this simply means we read the last value written to a variable).

4.1 Synchronization Actions

For the code below, what are the synchronization actions?

```
x = 0;
volatile ready = false;

Thread 1:       Thread 2:
x = 1;          if (ready) {
ready = true;     print(x)
                }
```

4.2 Reasoning using PO and SW

The code in 4.1 has two possible outcomes, according to our memory model: Nothing gets printed or a value gets printed. For the case that something gets printed, explain why it can only be the value 1. Use the happens-before order.

5 Conclusion

We have given you a glimpse into how practical memory models (i.e., those used in Java or C++) work. We have glossed over many details and were not very formal. If you want a one-sentence summary, remember that volatile variables synchronize with each other and thus give an ordering of statements accross threads.

If you feel cheated by the simplifications we made and would like to see a more formal description, look for the article "The Java Memory Model" by Manson, Pugh and Adve (we do not expect you to read/understand this, what we presented here is enough to perform all Java memory model reasoning you will be asked to do in the context of this lecture).